



3.4 自上而下语法分析

3.4.1 自上而下分析的一般方法

3.4.2 消除左递归

3.4.3 提取左因子

3.4.4 递归下降分析

3.4.5 预测分析器



3.4 自上而下语法分析

3.4.1 自上而下分析的一般方法

用**推导**的方法分析输入序列：（试探+回溯，**边推导边匹配**）

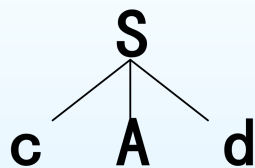
1. 对输入序列 ω ，从S开始进行**最左推导**，直到得到一个合法句子或非法结构；
2. **从左到右**扫描输入序列，试图用一切可能的方法，**自上而下**建立它的语法树；
3. 一种**试探**的过程，反复使用不同产生式，谋求与输入序列匹配；

[例3.16] 用下述文法分析输入序列 $\omega=cad$ ：

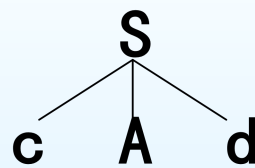
$S \rightarrow cAd$

$A \rightarrow ab$

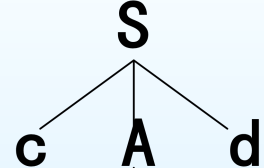
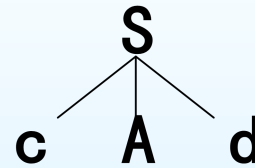
| a



$S \rightarrow cAd$



$A \rightarrow ab$



$A \rightarrow a$ 2

3.4 自上而下语法分析

问题：

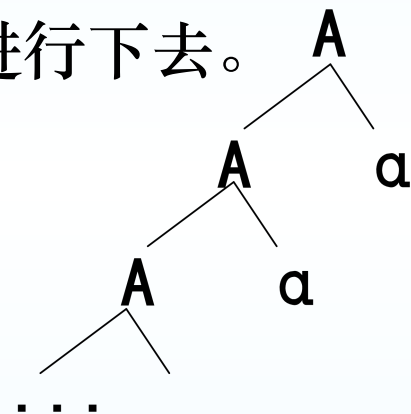
- ① 若有 $A \rightarrow \alpha\beta_1 | \alpha\beta_2$ ，**公共左因子**，则会造成虚假匹配和大量回溯；造成分析效率低、语义动作难以恢复、以及出错位置的报告不确切等。
- ② 若有 $A \rightarrow A\alpha$ ，**左递归**，则死循环使分析无法进行下去。 $\hat{A}$

- ② 若有 $A \rightarrow A\alpha$ ，左递归，则死循环使分析无法进行下去。

重写文法：

- ① 消除左递归，以避免陷入死循环；
- ② 提取左因子，以避免回溯。

- ② 提取左因子，以避免回溯。



3.4.2 消除左递归

定义3.9 若文法G中的非终结符A，对某个文法符号序列 α 存在推导 $A \Rightarrow^+ A\alpha$ ，则称G是左递归的。若G中有形如 $A \rightarrow A\alpha$ 的产生式，则称该产生式对A直接左递归。



3.4 自上而下语法分析

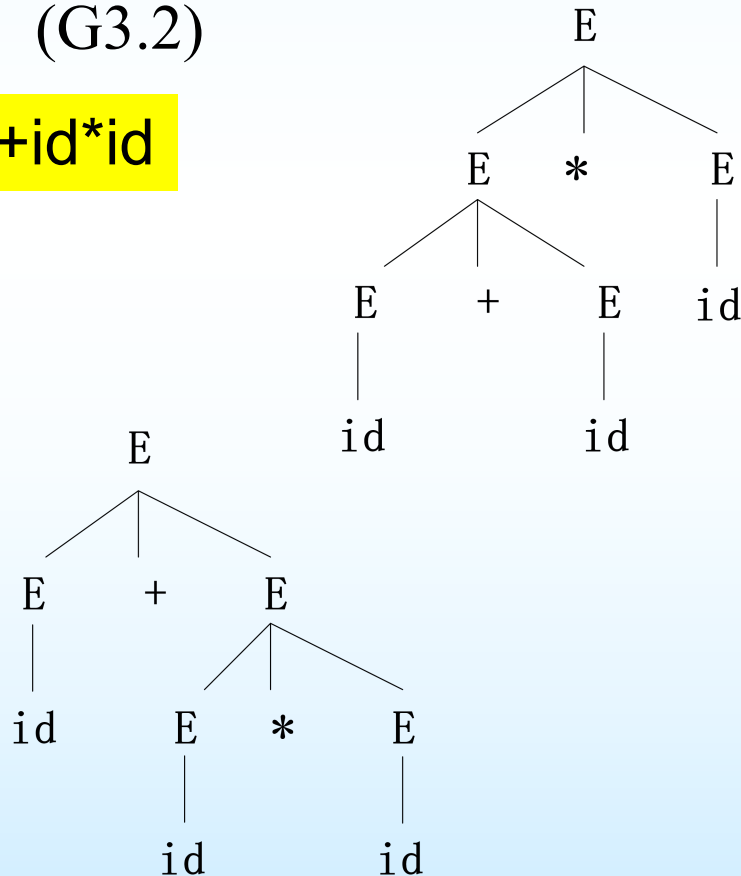
[例] 消除算术表达式文法的二义性，得到有直接左递归的文法：

$E \rightarrow E + E \mid E * E$

$\mid (E) \mid -E \mid id$

(G3.2)

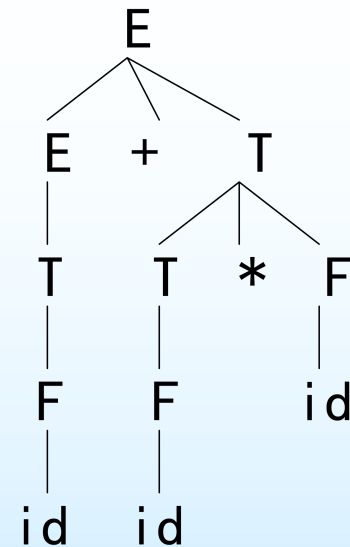
id+id*id



$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$ (G3.4)

$F \rightarrow (E) \mid -F \mid id$





3.4 自上而下语法分析

1. 消除文法的直接左递归

考虑仅有两个候选项: $A \rightarrow A\alpha \mid \beta$ 产生的句型: $\beta\alpha^*$

替换为: $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$ 消除了一个直接左递归

二者区别: 句型中文法符号的生成顺序不同。后者的生成顺序与对输入序列从左到右的扫描顺序相同。

算法3.1 消除直接左递归

输入 G中所有的A产生式(含直接左递归)

输出 等价的不含直接左递归的G'

方法 首先, 整理A产生式为如下形式:

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

其中 α_i 非空, β_j 均不以A开始。用下述产生式代替A产生式

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon \text{ (不要忘记}\epsilon, \text{否则推导无法终止)}$$



3.4 自上而下语法分析

[例3.17] 消除算术表达式文法的直接左递归:

$$E \rightarrow E + E \mid E * E \quad E \rightarrow E + T \mid T \quad E \rightarrow TE' \quad (1)$$

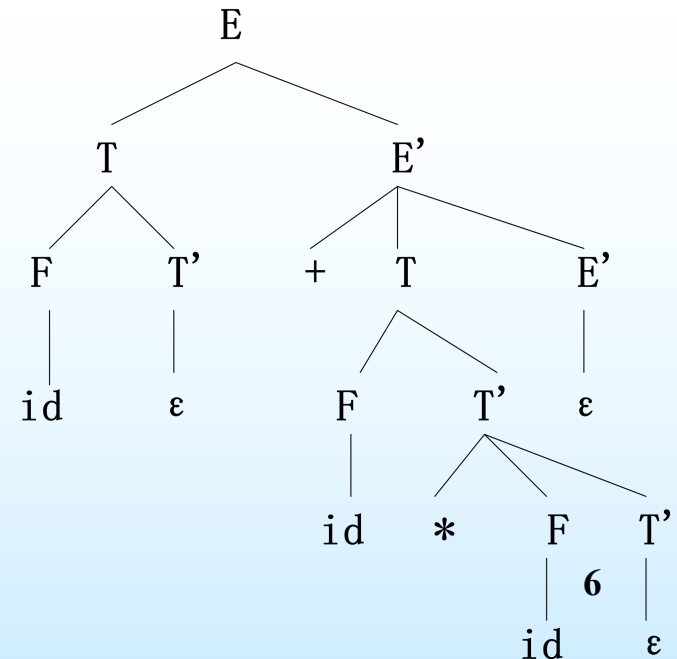
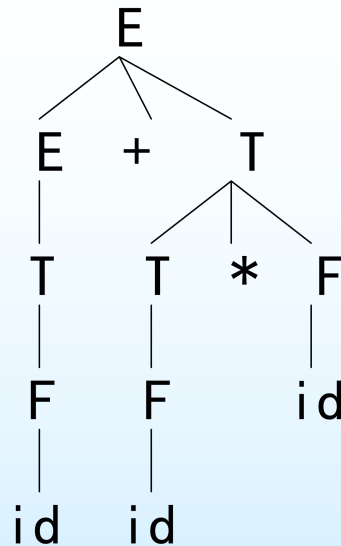
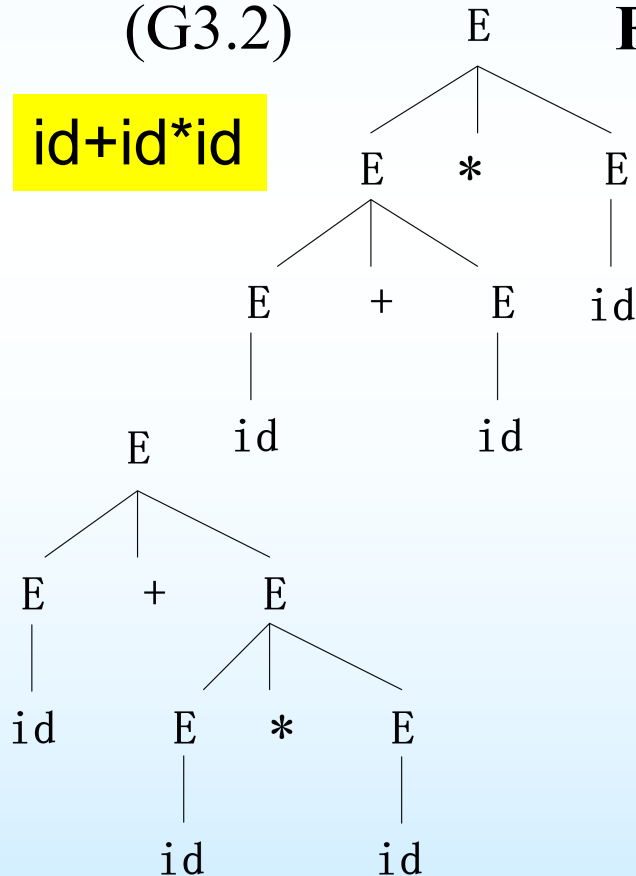
$$\mid (E) \mid -E \mid id \quad T \rightarrow T * F \mid F \quad (G3.4) \quad E' \rightarrow +TE' \mid \epsilon \quad (2)$$

$$(G3.2) \quad F \rightarrow (E) \mid -F \mid id \quad T \rightarrow FT' \quad (3) \quad (G3.4')$$

$$T' \rightarrow *FT' \mid \epsilon \quad (4)$$

$$F \rightarrow (E) \mid -F \mid id \quad (5)..(7)$$

id+id*id



3.4 自上而下语法分析

2. 消除文法的左递归 (直接/间接左递归)

算法3.2 消除左递归

输入 无回路文法G (不出现 $A \Rightarrow^+ A$)

输出 无左递归的等价文法G'

方法 将(含左递归的)非终结符合理排序: $A_1, A_2, \dots, A_n$;

for i in 1..n

loop for j in 1..i-1

loop

用 $A_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ 的右部替换每个形如 $A_i \rightarrow A_j \gamma$ 产生式中的 A_j ,
得到新产生式: $A_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$; /*先消除 A_j 间接左递归*/
end loop;

(如果有)消除 A_i 产生式中的直接左递归; /*再消除直接左递归*/

end loop;

核心思想: 将不是直接左递归的非终结符右部展开到其他产生式中

注意: 若G产生句子的过程中出现 $A \Rightarrow^+ A$, 则无法消除左递归

G3.8: 直接+间接左递归

$S \rightarrow Aa \mid b$

$A \rightarrow Ac \mid Sd \mid \varepsilon$

$S \Rightarrow Aa \Rightarrow Sda$

G': 只含间接左递归

$A \rightarrow Bcd$

$B \rightarrow Ce \mid f$

$C \rightarrow Ab \mid c$

$A \Rightarrow Bcd \Rightarrow Cecd \Rightarrow Abecd$



3.4 自上而下语法分析

核心思想：将不是直接左递归的符号右部展开到其他产生式

关键步骤：合理排序非终结符： $A_1, A_2, \dots, A_n$; (排序方法不唯一)

用 $A_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ 右部替换 $A_i \rightarrow A_j \gamma$ 中的 A_j ，得到
 $A_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$;

(如果有)消除 A_i 产生式中的直接左递归;

[例3.18] 用算法3.2消除文法 $S \rightarrow Aa | b \quad A \rightarrow Ac | Sd | \varepsilon$ 中的左递归。

① 将 S 的右部展开在 A 中，得到：

$A \rightarrow Ac | Aad | bd | \varepsilon$ (消除 S 间接左递归)

② 消除新产生式中的直接左递归，得到：

$S \rightarrow Aa | b$

$A \rightarrow bdA' | A'$ (G3.8')

$A' \rightarrow cA' | adA' | \varepsilon$



3.4 自上而下语法分析

3.4.3 提取左因子

$$S \rightarrow cAd$$

$$A \rightarrow ab \mid a$$

当不确定用A产生式的哪个候选项替换A时，可以重写A产生式来**推迟**这种决定，直到看见足够的输入，能正确决定所需选择为止。这一过程被称为**提取左因子**，它**类似于有限自动机的确定化**。

将： $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

替换为： $A \rightarrow \alpha A' \quad A' \rightarrow \beta_1 \mid \beta_2$



3.4 自上而下语法分析

算法3.3 提取文法的左因子

输入 文法G

输出 等价的无左因子文法G'

方法 重复过程，直到所有A产生式候选项中不再有公共前缀：
重排A产生式： $A \rightarrow \alpha\beta_1 | \alpha\beta_2 | \dots | \alpha\beta_n | \gamma$ ；并用 $A \rightarrow \alpha A' | \gamma$ 和 $A' \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$ 取代原A产生式。

[例3.20] 考察悬空else文法： $S \rightarrow iCtS \mid iCtSeS \mid a$ $C \rightarrow b$

用算法3.3提取左因子，得到如下文法：

$$S \rightarrow iCtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$C \rightarrow b$$

既有左递归又含左因子？
可以先消除左递归。



3.4 自上而下语法分析

[例3.19] 算术表达式组成的语句序列，文法G3.9如下：

$$L \rightarrow E;L \mid \varepsilon$$

$$E \rightarrow E+T \mid E-T \mid T$$

$$T \rightarrow T*F \mid T/F \mid T \bmod F \mid F$$

$$F \rightarrow (E) \mid \text{id} \mid \text{num}$$

消除左递归后的等价文法G3.9'如下：

$$L \rightarrow E;L \mid \varepsilon$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid -TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid /FT' \mid \bmod FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id} \mid \text{num}$$

既是直接左递归又是左因子？

可以先消除左递归。

因为消除左递归后，左因子也随之消失。



3.4 自上而下语法分析

[例] 文法G如下:

$A \rightarrow Bcd \mid B$

$B \rightarrow Ce \mid f$

$C \rightarrow Ab \mid c$

消除左递归时, 可尝试排序**BAC**或**ACB**。
按不同的顺序消除左递归和提取左因子得到的文法可能不同。但要保证文法开始符不变。

先消除左递归再提取左因子后的等价文法如下:

排序: CBA

(1)将C展开在B的右部

$B \rightarrow Abe \mid ce \mid f$

(2)将B展开在A的右部

$A \rightarrow Abecd \mid cecd \mid fcd \mid Abe \mid ce \mid f$

(3)消除A的直接左递归

$A \rightarrow cecdA' \mid fcdA' \mid ceA' \mid fA'$

$A' \rightarrow becdA' \mid beA' \mid \epsilon$

(4)提取A和A'中的左因子

$A \rightarrow ceD \mid fD$

$A' \rightarrow beD \mid \epsilon$

$D \rightarrow cdA' \mid A'$

B和C的产生式无用了。



3.4 自上而下语法分析

3.4.4 递归下降分析

- ① 自上而下分析是一种边推导边匹配的过程，推导意味着对非终结符的展开，对输入序列的匹配是从左到右进行的；
- ② 递归下降分析是**自上而下分析**的一种实现方式，是直接**以程序的方式**模拟产生式产生语言的过程；
- ③ 它对文法的**限制是不含公共左因子和左递归的非二义文法**；
- ④ 基本思想：（**产生式**）每个非终结符对应一个子程序，子程序的过程体按产生式候选项分情况展开。（**推导**）出现在产生式右边的非终结符对应子程序调用；（**匹配**）终结符则与输入序列匹配；
- ⑤ 它是一种非形式化的方法，只要能写出子程序，用什么样的方法和步骤均可。

一种稳妥的方法

- ① 构造文法的状态转换图并且化简；
- ② 将转换图转化为EBNF表示；（EBNF与程序结构存在对应关系）
- ③ 从EBNF构造子程序。



3.4 自上而下语法分析

1. 构造状态转换图且化简

递归下降分析的文法:

$$L \rightarrow E ; L \mid \varepsilon$$

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid T \bmod F \mid F$$

$$F \rightarrow (E) \mid \text{id} \mid \text{num}$$

消除左递归后的等价文法:

$$L \rightarrow E ; L \mid \varepsilon$$

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid - T E' \mid \varepsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid / F T' \mid \bmod F T' \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id} \mid \text{num}$$

(1.1) 每个非终结符对应一个状态转换图:

- ① 为非终结符A建立一个初态和一个终态;
- ② 为 $A \rightarrow X_1 X_2 \dots X_n$ 构造从初态到终态的路径, 边标记为 $X_1, X_2, \dots, X_n$ 。
- ③ 根据识别同一集合的原则, 化简转换图。



3.4 自上而下语法分析

从初态到终态的每条路径都表示一个产生式。

$L \rightarrow E ; L \mid \epsilon$

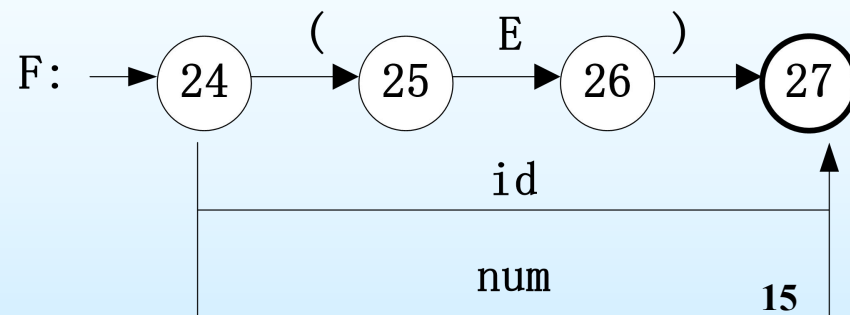
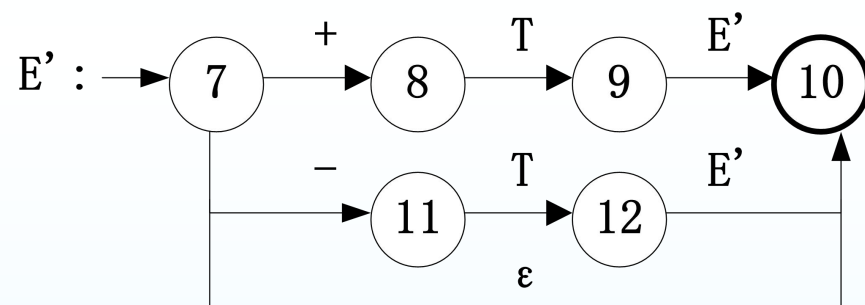
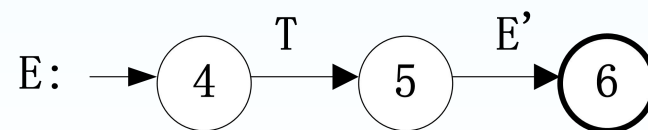
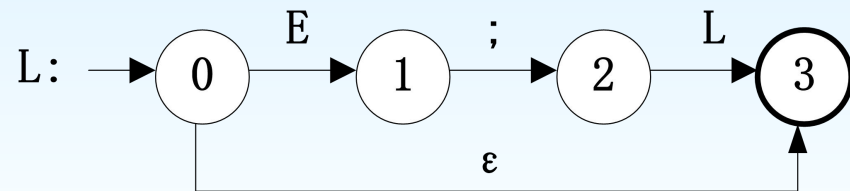
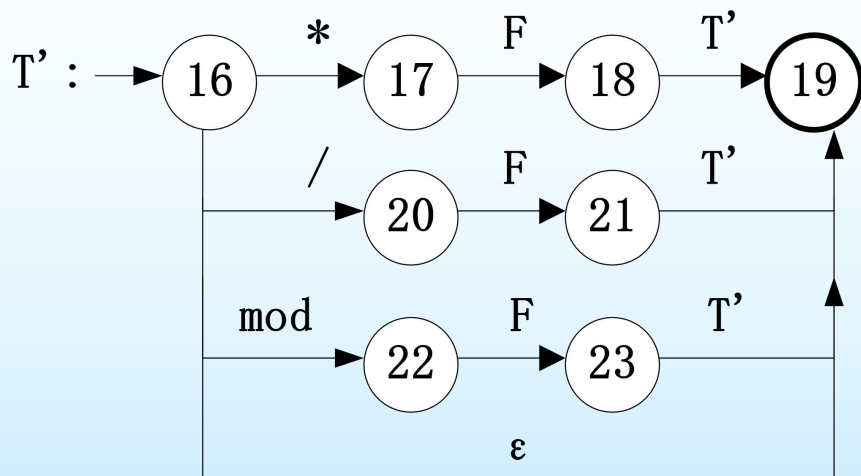
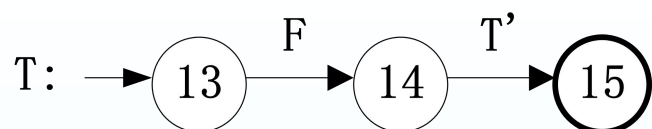
$E \rightarrow T E'$

$E' \rightarrow + T E' \mid - T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid / F T' \mid \text{mod } F T' \mid \epsilon$

$F \rightarrow (E) \mid \text{id} \mid \text{num}$

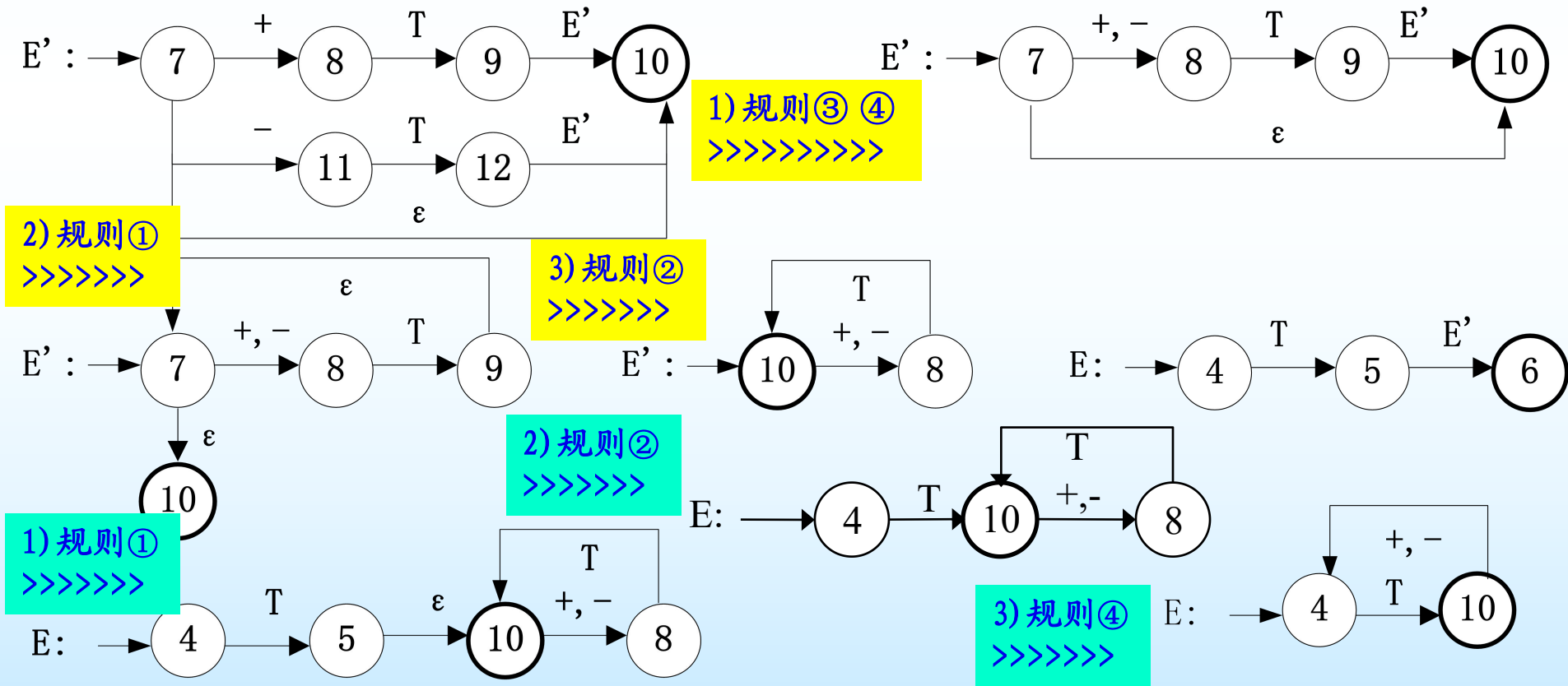




3.4 自上而下语法分析

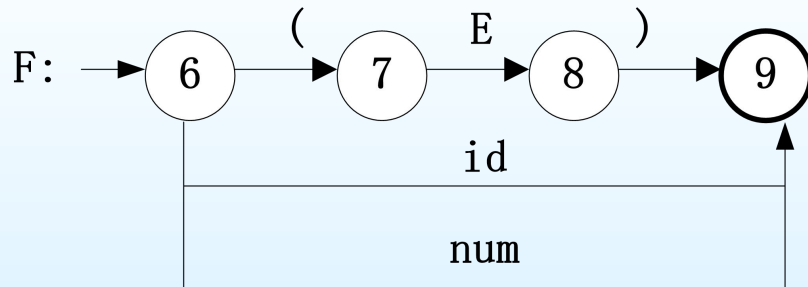
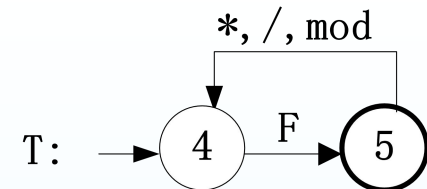
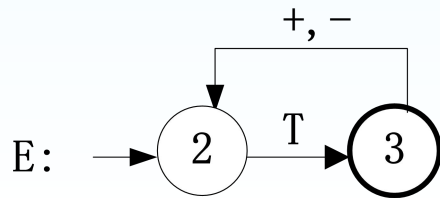
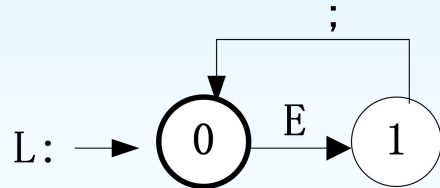
(1.2) 状态图的化简原则

- ① 标记为A的边可等价为标记 ϵ 的边转向A转换图的初态(与子程序的关系);
- ② ϵ 边连接的两个状态可以合并;
- ③ 标记相同的路径可以合并;
- ④ 不可区分的状态可以合并。





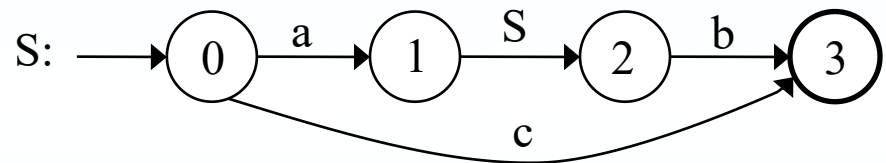
3.4 自上而下语法分析



注意:

$$L = \{a^n c b^n \mid n \geq 0\}$$

G: $S \rightarrow aSb \mid c$



不能用规则①化简S的状态转换图；
这样的状态转换图对应子程序中的
直接递归调用（调用自己）。

间接递归调用
 $E \rightarrow T \rightarrow F \rightarrow E$



3.4 自上而下语法分析

2. 文法的扩展BNF (EBNF) 表示

① $\{ \}$: 重复0或若干次 (while)

② $[]$: 可选择 (if或while)

③ $|$: 括弧()内的或关系 (case)

④ $()$: 括号内的成分优先

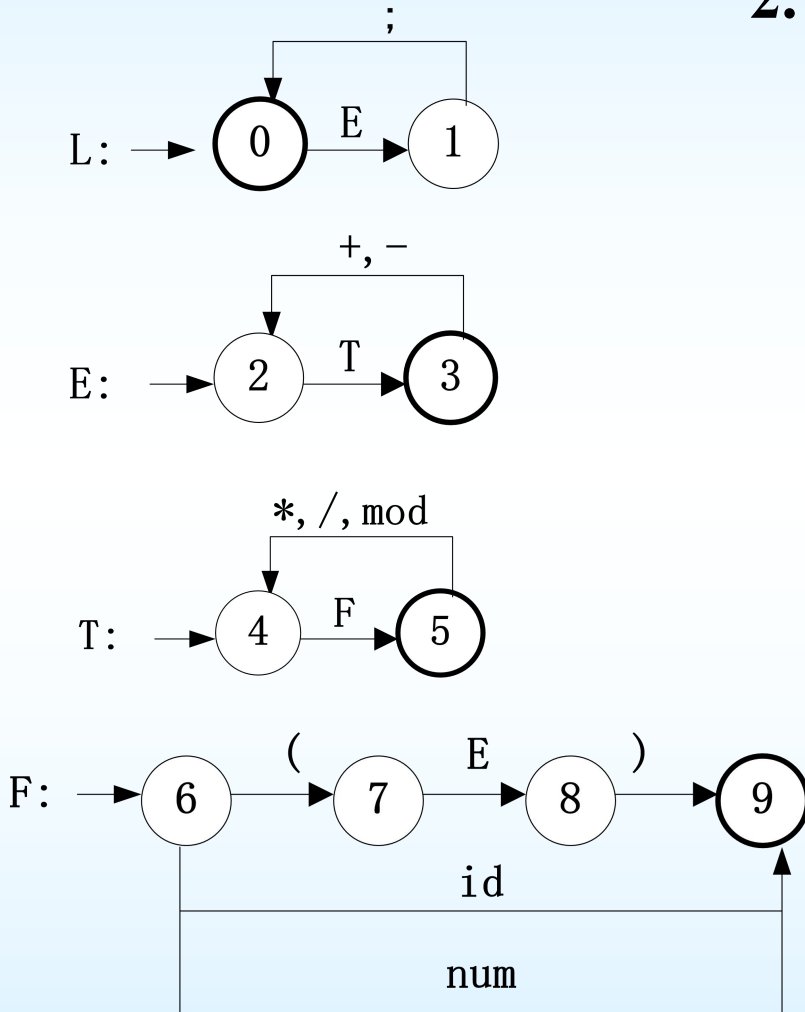
EBNF表示:

$L \rightarrow \{ E; \}$

$E \rightarrow T \{ (+ | -) T \}$

$T \rightarrow F \{ (* | / | \text{mod}) F \}$

$F \rightarrow (E) | \text{id} | \text{num}$



3.4 自上而下语法分析

$$L \rightarrow \{ E; \}$$
$$E \rightarrow T \{ (+ | -) T \}$$
$$T \rightarrow F \{ (* | / | \text{mod}) F \}$$
$$F \rightarrow (E) | \text{id} | \text{num}$$

3. 递归下降子程序

procedure L is

begin

lookahead := lexan; //调用词法分析器，返回一个终结符。lookahead保存输入序列中当前待匹配的终结符

while (lookahead!=eof) loop **E; match(';');** end loop;

end L;

procedure E is

begin

T;

while lookahead ∈ (+|-)

loop match(lookahead); T;

end loop;

end E;

T的子程序见书。

procedure match(t) is

begin

if t=lookhead

then lookahead:=lexan;

else error(“syntax error1”);

end if

end match;

3.4 自上而下语法分析

$$L \rightarrow \{ E; \}$$
$$E \rightarrow T \{ (+ | -) T \}$$
$$T \rightarrow F \{ (* | / | \text{mod}) F \}$$
$$F \rightarrow (E) | \text{id} | \text{num}$$

3. 递归下降子程序

procedure L is

begin

lookahead := lexan; //调用词法分析器，返回一个终结符。lookahead保存输入序列中当前待匹配的终结符

while (lookahead!=eof) loop **E; match(';');** end loop;

end L;

procedure E is

begin

T;

while lookahead ∈ (+|-)

loop match(lookahead); T;

end loop;

end E;

T的子程序见书。

procedure F is

begin

case lookahead is

'(' : match('('); E; match('');

id : match(id);

num : match(num);

others : error("syntax error2");

end case;

end F;



3.4 自上而下语法分析

如果**不消除左递归**：

若存在产生式 $E \rightarrow E + id$ ，则E的递归下降子程序如下：

```
procedure E is
```

```
begin
```

```
    E;
```

```
    match('+');    ——永不执行
```

```
    match(id);     ——永不执行
```

```
end E;
```

此程序永不停机。

同样，文法中的**公共左因子**也会给递归下降分析造成困难。